Pthreads

Real-time systems TP3, TP4, and TP5

Lab objective

Write a real-time program with *pthreads* library.

You can work in teams of **two**.

Tools:

Linux with root account (*GEI* 101, 103, 105, and 107). As the rooms are not in free-access you can use your own machines with Linux.

Webots (https://cyberbotics.com/). Webots are already installed on rooted INSA machines (*GEI* 101, 103, 105, and 107). To install on your own machine follow the instructions from: https://cyberbotics.com/doc/guide/installation-procedure

Due date: end of TP5

Other Materials:

- Lecture 7

Lab objectives:

- Create periodic task
- Use semaphore to activate sporadic task
- Select scheduling policy and assign priorities
- Protect critical sections with mutexes and resource access protocols
- Design a simple control application with periodic/sporadic tasks

Steps:

1 Pre-project (5pts)

2 Mini project (10pts)

1 Pre-project

If you are using INSA machine, choose "*TP GEI* 2024-25 j" boot option.

If you are using your own machine, you will need to either run certain processes as a *root* or give the real-time scheduling privileges to the *user*:

```
/etc/security/limits.conf
@user  -  rtprio  99
@user  -  nice   -20

/etc/security/capability.conf
cap_sys_nice            user
```

You can use *mlockall* to ensure that all pages mapped currently (*MCL_CURRENT*) and in the future (*MCL_FUTURE*) will be locked.

```
if(mlockall(MCL_CURRENT|MCL_FUTURE) == -1) {
        printf("mlockall failed\n");
        exit(-2);
}
```

Start from a simple C or C++ program.

- Write a periodic real-time task that displays response times of its jobs. You can add a generic load to your task:

```
for(int i=0; i<load*1000; i++){
 /* do nothing, keep counting */
}
```

- Create two periodic tasks and observe their response times. Tasks should have different periods (*e.g.,* 1 and 2 seconds). Make sure that the tasks start synchronously (*e.g.,* use *pthread_barrier_t*).

- Assign both tasks to the same processor. To get the list of available processors use $ cat /proc/cpuinfo | grep processor. To run a process on a selected processor or a set of processors use `taskset` command.

- Observe the response time of each task. Then, execute both tasks together in the following configurations:

  – both tasks use *SCHED_RR* and have the same priority

  - both tasks use *SCHED_RR* and task 1 has higher priority

  Compare the observed response times. Explain the differences. If there are no differences, increase execution time of one or both tasks. Is the response time of the lower priority task always the same?

- Add a third task with a mid-priority level. Add critical sections protected with a mutex to the tasks with highest- and lowest-priority. Try to find the timing parameters that will lead to priority inversion (Lecture 6). Next, use *Priority Inheritance Protocol* to remove the priority inversion.

2 Project

Based on your pre-project, write a simple real-time application of your choice using *pthread* library. The application should contain:

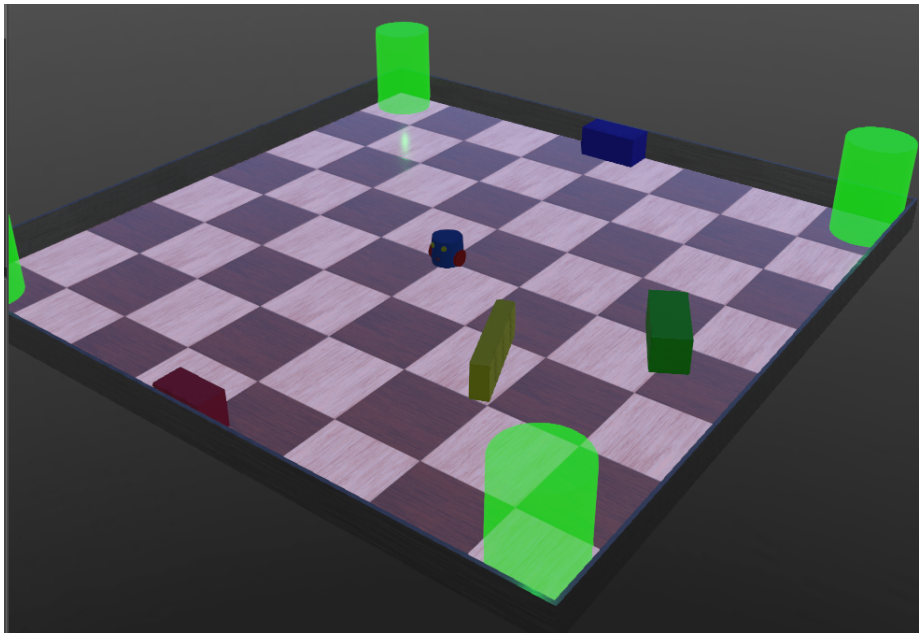
- at least two periodic or sporadic tasks with different priorities,
- a shared resource protected with a mutex and resource access protocol,
- a semaphore to trigger event-driven actions.

For example, we can propose to write a real-time application for remotely controlled robot using Webots simulation tool or a server for multiple clients that send periodically computational requests.

The control application connects to the robot via the TCP/IP server. Its basic functionality could be:

- Drive robot from wall to wall (high-priority)
- Display on console the batter level (low-priority)

Additionally, we can consider object avoidance and battery recharging.



If you are working on INSA machines, connect to the Internet (open login page in Firefox).

Download Webots INSA world and open *Webots*. Use *taskset* to run *Webots* on different processors than the ones your application will run.

message id	**arguments**	**example**	**description**
M	v_L and v_R	$M, 50, 50$	set speed of left and right motor to $v_L \cdot v_{max}/100$ and $v_R \cdot v_{max}/100$
S	-	S	read distance sensor values as S, d_L, d_R where d_L and d_R are the readings from the left and right distance sensor
W	-	W	read wheel sensor values as S, w_L, w_R where w_L and w_R are the readings from the left and right wheel sensor
B	-	B	read battery level as B, b_L where b_L is the battery level
T	α	$T, 1.57$	turn robot to α where $\alpha \in [-\pi, \pi]$; message id T will be sent back when robot will achieve the requested position.

If needed, the controller can be modified in /controllers/my_controller.c

Examples:

- Setting both motors to 50%:

```
send(fd,"M,50,50\n",strlen("M,50,50\n"),0);
```

- Reading distance sensors:

```
double left_sensor, right_sensor;
char buffer[256];
send(fd,"S\n",strlen("S\n"),0);
int n = recv(fd, buffer, 256, 0);
buffer[n] = '\0';
printf("%s\n",buffer);
sscanf(buffer,"S,%lf,%lf", &left_sensor, &right_sensor);
```

- U-turn

```
send(fd,"T,3.14\n",strlen("T,3.14\n"),0);
int n = recv(fd, buffer, 256, 0);
```

or

```
send(fd,"T,0.00\n",strlen("T,0.00\n"),0);
int n = recv(fd, buffer, 256, 0);
```

add SO_REUSEADDR